

● TSBM 2026

JVM vs V8

비슷하면서도 다른!

소개

- 디씨클 - 플랫폼 개발, 검색, RAG, Agent, Data Engineering, ...

이번 발표의 지향점

- jvm과 v8이 유사한 점과 차이점을 보면서 원리를 더 쉽게 장기 기억에 새기자.
- 다룰 주제들:
 - execution context vs java stack frame 방식
 - 컴파일/인터프리팅/JIT 방식
 - 이벤트루프와 가상 스레드
 - gc 방식
 - 마지막으로, 실무적 차이들.

첫 번째, 콜스택 비교

먼저 실제 실행 중 함수 호출이 어디에 쌓이고,
스코프와 지역 변수가 어디에 보관되는지 봅시다.

- JS: Execution Context + Lexical Environment
- Java: Stack Frame + Local Variables + Operand Stack

js Execution Context + Lexical Environment

EXAMPLE CODE

```
const rate = 0.1;
function checkout(amount) {
  const fee = 5;
  function price() {
    const tax = amount * rate;
    return amount + tax + fee;
  }
  return price();
}
checkout(100);
```

CALL STACK

EC 3

price()

Function EC

LexicalEnvironment → LE 3



EC 2

checkout(100)

Function EC

LexicalEnvironment → LE 2



EC 1

global

Global EC

LexicalEnvironment → LE 1



LEXICAL ENVIRONMENTS

LE 3

price Lexical Env

bindings

tax=10

outer

→ checkout Lexical Env

LE 2

checkout Lexical Env

bindings

amount=100, fee=5, price=function

outer

→ Global Lexical Env

LE 1

Global Lexical Environment

rate

0.1

checkout

function object

outer

null

아래에서 위로 EC가 쌓이고, 각 EC는 같은 번호의 Lexical Env를 참조한다. 스코프 체인은 LE의 outer 참조로 이어진다.

java Stack Frame

SAME SHAPE IN JAVA

```
static int checkout(int amount) {
    int fee = 5;
    return price(amount, fee);
}
static int price(int amount, int fee) {
    int rate = 10;
    int tax = amount * rate / 100;
    return amount + tax + fee;
}
public static void main(String[] args) {
    checkout(100);
}
```

THREAD STACK

SF 3

price(100, 5)

Stack Frame

local variables + operand stack

SF 2

checkout(100)

Stack Frame

waiting for return value

SF 1

main()

Stack Frame

entry point

FRAME CONTENTS

SF 3

price() Stack Frame

Local Variables	amount=100, fee=5, rate=10, tax=10
Operand Stack	amount, tax, iadd, fee, iadd
Method Ref	price(int,int):int
Return Address	checkout()



SF 2

checkout() Stack Frame

Local Variables	amount=100, fee=5
Operand Stack	price return value를 기다림



SF 1

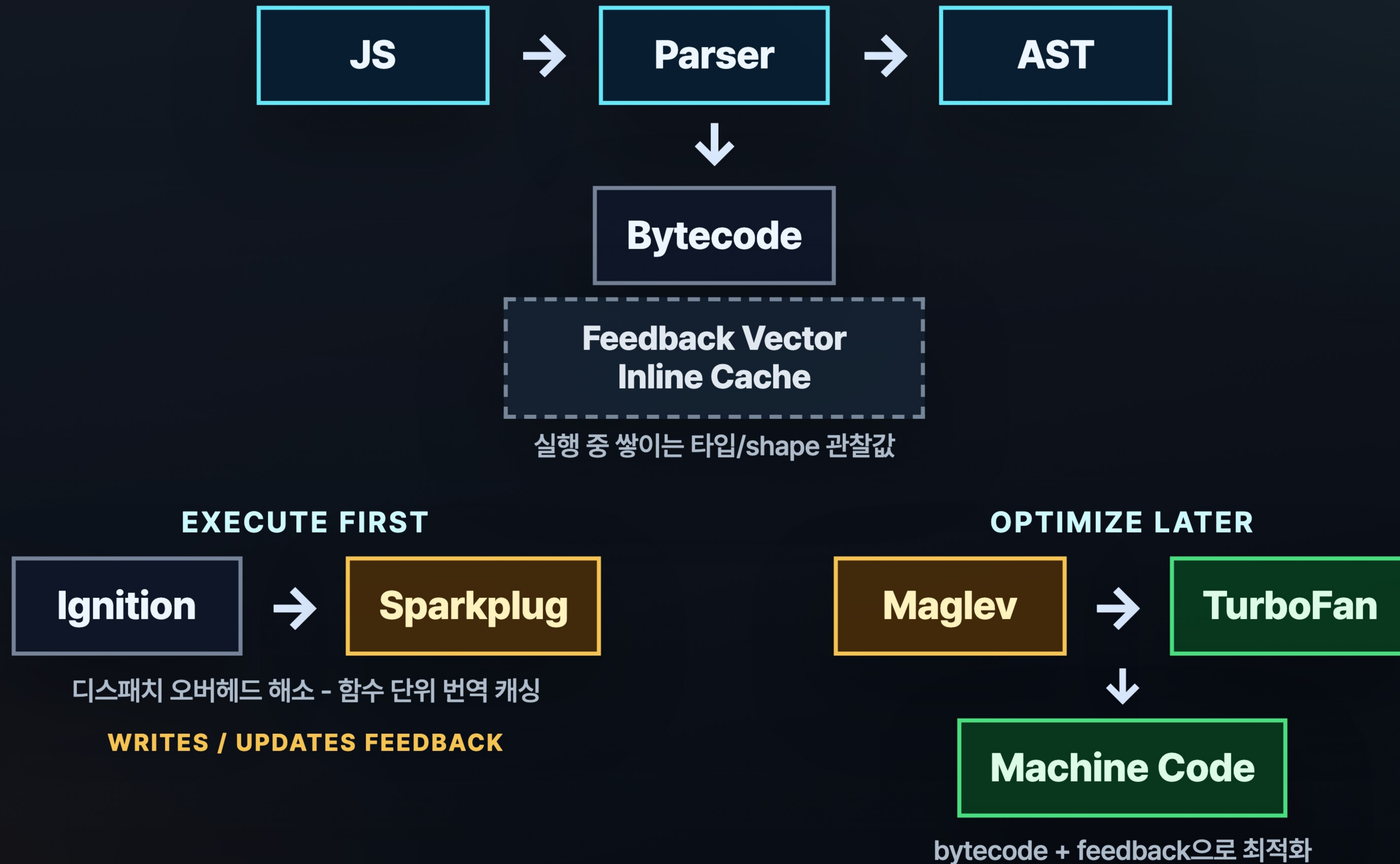
main() Stack Frame

Local Variables	args
next	checkout() return 후 계속 실행



Java도 아래에서 위로 Stack Frame이 쌓인다. 다만 별도 Lexical Env 객체가 아니라 각 프레임 안의 local variable table과 operand stack이 중심이다.

두 번째, 컴파일러/인터프리터/JIT



컴파일과 인터프리팅은 정말로 뭘까?

- 컴파일: 하나의 언어에서 다른 언어로 번역 (TS => JS, JS => 바이트코드, Java => 바이트코드)
- 인터프리팅은 그러면 뭘지? => 면접에서 다들 하는 말: 한 줄 한 줄씩 읽는다.

```
while (running) { // 인터프리팅은 디스패치 테이블에서 while - switch를 하는거라고 생각하는게 매우 편하다.
    uint8_t op = bytecode[ip++];
    // 🚀 [여기가 핵심!] CPU가 op 번호를 보고 점프 테이블(Jump Table)을 확인합니다.
    switch (op) { // "op가 ADD네? 메모리에 미리 컴파일되어 있는 case ADD 구역으로 점프!"
        case LOAD:
            // ---> (도착 지점 1)
            // 아래 코드는 이미 C 컴파일러에 의해 OS/하드웨어에 맞는 기계어로 변환되어 있습니다.
            // 점프해 온 CPU는 여기서부터 순차적으로 기계어를 실행합니다.
            push(locals[bytecode[ip++]]);
            break; // 실행이 끝나면 다시 루프의 처음(while)으로 돌아갑니다.

        case ADD:
            // ---> (도착 지점 2)
            // 마찬가지로 점프해서 도착한 후, 아래의 기계어들을 순식간에 차례대로 실행합니다.
            b = pop();
            a = pop();
            push(a + b);
            break;
            // ...
    }
}
```


왜 굳이 바로 기계어로 안하고? - java

- 1995년 ~ 1999년 java가 처음 등장했을 때: 아 이거 OS랑 CPU마다 매번 기계어 번역 다르게 번역해야돼. 짜증나.
- 하나의 추상화된 계층을 뒤. 배열(스택)은 자료구조니까 OS 상관없이 만들면 다 똑같음.
- 전보다 호환성이 좋아지고 그 외에도 몇 가지 장점이 존재했음.
- 1999년 문제 - 너무 느리다: 아 이거 반복되는 부분은 더 fit한 기계어로 최적화 해줘야겠는데? => 이는 jit이라고 부른다.

왜 굳이 바로 기계어로 안하고? - js

- 1995 ~ 2008 V8 이전: 엔진마다 AST walking 인터프리터, bytecode 인터프리터, 초기 JIT 등이 섞여 발전
 - js가 아주 조금씩만 사용되던 때. 그만큼 가볍고 나이브한 접근 법.
- 2008 ~ 2016/2017 V8: Full-codegen + Crankshaft => 실제로 빠르게 기계어로 먼저 했음!
 - 스마트폰 문제 - 기계어 메모리 부족
 - 구글맵/지메일 => js 코드량이 늘어나! => 기계어도 더 많아! => OOM
- 2016/2017 이후 V8: Ignition + TurboFan
 - 더 더 복잡한 JS 문법과 늘어나는 코드량에 대응하자!
- 정리하면:
 - Java의 중간 계층은 호환성/이식성이 핵심 출발점이었다.
 - V8의 구조 변화는 브라우저 환경에서 빠른 실행, 빠른 시작, 낮은 메모리 사용량, UI 반응성을 맞추기 위한 선택과 진화였다.

JIT



- 역치값: java 1500/10000 => 2000/15000 vs js 예산 시스템 (feedback vector) [+OSR과 933%, 107%]
- C1 t1 vs Sparkplug - 디스패치 오버헤드 해소 + alpha
 - t1의 주요 최적화 (45~55%) - 메서드 인라이닝
- C1 t2~t3 vs Maglev - 준비 vs 투기
 - maglev의 타입 세금 => 증명까지의 시간단축 => 투기 => 그리고 역최적화
- C2 (t4) vs Turbofan - 각자 full 증명, full 최적화

v8 - 아까 역사에서 이어서.

- v8은 2008 ~ 2016년 동안은 full code였다. 브라우저가 받자마자 바로 최적화된 기계어로 번역.
 - js 양이 적었다.
- 2016년에 스마트폰 메모리 문제로 인터프리터 도입.
 - 이때 도입된 인터프리터 이름이 ignition이다. turbofan도 이때 같이 도입되었다.
 - 현대에는 js코드양이 워낙 많고 바로 들어가자마자 반응해줘야해서 자연스럽게 이 기조가 유지중.
- 2021년에는 Sparkplug가 추가되었다.
- 2023년에는 Maglev(자기부상열차)가 나왔다.
 - 스파크플러그와 터보팬 사이의 간극을 매꿔줄 히어로.
 - 이로써 jvm과 굉장히 유사한 시스템으로 자리매김한다.

jvm - 아까 역사에서 이어서.

- 1995 ~ 1998 jvm은 끔찍하게 느리다. JIT가 없음. jvm의 등장 목적은 OS 상관없이 코드짜자 => 이게 기조였음.
- 1999 hotspot jvm의 등장. 드디어 just-in-time 컴파일링이 가능해졌다. -> 근데 C1 C2가 자동이 아니라 둘 중 하나만 선택 가능했음.
- 2011 ~ 2014 C1 ~ C2 계층 컴파일의 도입. => v8보다 10년 빠름.

다시 돌아와서 비교

- 둘 다 결국 계층적으로 바이트코드를 최적화하는 구조에 도달했다. (바이트코드는 jvm, v8 각자 독자적으로 몇 백개씩 정해져있음.)
- 우리가 쓰는 코드는 jit으로 기계어로 최적화가 되지만, 반복이 많을 때만 된다.
- 이게 개발할 때 무슨 도움이 되느냐? 흠... 딥다이브 트러블 슈팅에서 도움 받는 사례들은 종종 보인다.
- 평상시에 이런 것도 설명할 수 있다:
 - 왜 어떤 함수는 빨라지고 어떤 함수는 느려지는가? 동적 타입이 바뀌면 deopt된다.
 - jvm은 좀 더 헤비하게 최적화하기에 워업 시간이 길다.
 - node는 더 빠르게 띄우고 가볍게 최적화해두는 브라우저 환경에서 선제작된 v8을 쓰기 때문에 워업은 빠르고 시간 지나면 좀 더 느리다.
 - 즉, 최고속도는 jvm이 더 좋다. 제로백은 js가 더 좋다.
 - 사실 js는 동적 타입 때문에 최적화가 더 어려운 것도 크다고 한다. v8의 "good enough"가 보이는 지점.

3번째, 이벤트루프와 가상스레드

- jvm -> 이제 가상 스레드를 지원함. -> 이게 여러개의 스택 프레임 위에 mount/unmount를 반복 -> 하나의 프로세스 내에서 모두 처리.
 - 이전부터 단일 프로세스에서 진화함. c10k 문제 때부터. - connection pool로 진화 - 이제 가상 스레드.
 - 네트워크 I/O는 NIO/epoll 계층에서 가볍게 받을 수 있었지만, JDBC 같은 blocking API가 platform thread를 붙잡는 병목이 남아있었다.
 - 가상 스레드는 thread-per-request 모델을 유지하면서, blocking I/O 대기 중 carrier thread를 반납하게 해준다.
- v8 -> 이벤트 루프. 가볍게 단일 스레드. -> 한계 극복 위해 여러 프로세스 병렬로 띄워서 스케일링. (pm2, Cluster)
- 하나의 프로세스 vs 여러개의 프로세스의 차이.
 - "만약에" 기술적 선택을 한다면 결국 다중 프로세스 vs 단일 프로세스의 장단점도 따라다님.
 - 메모리 공유를 통한 자원 활용 vs 격리성에 의한 안정성
- 이런 현상은 바로 뒤에 나오는 gc에서도 유사한 양상을 띠م.

4번째, GC 비교

이번에는 함수 호출이 아니라, 객체가 쌓이고 사라지는 방식을 봅니다.

- 둘 다 generational GC를 쓴다.
- JVM은 큰 힙과 장기 실행 서버를 강하게 의식한다.
- V8은 브라우저 렌더링과 짧은 pause를 강하게 의식한다.

같은 GC라도 고향이 다르면 튜닝 목표가 달라진다.

gc 동시성 비교 - JVM collectors + V8 Orinoco

- serial, parallel, cms => 잘 안씀. / g1gc => 표준. / zgc는 jvm의 역할을 잘 보여줌
- orinoco는 cms와 상당히 비슷하지만, cms는 old가 하나의 큰 연속 공간이어서 compaction이 훨씬 까다로웠음.
 - g1과도 대비되는 부분. cms는 full gc에서만 compaction.



orinoco와 g1gc 나란히 놓고 보기

- JVM G1GC의 목적: 스루풋을 너무 훼손시키지 않으면서 gc.
 - young gc: 짧은 stw - parallel로 eden => survivor region으로 옮기기
 - old gc: concurrent marking => 가비지가 많은 old region을 young gc때 같이 청소. => 타킷 ms 최대한 맞춰서 진행.
 - 리전: 2048개로 나눔. 리전 하나당 1MB ~ 32MB. 힙 크기는 수 GB ~ 수십 GB. => 연속 공간
- V8 Orinoco의 목적: 기껏해야 수백 메가바이트(MB) ~ 1.4GB 남짓한 메모리(힙)에서 60fps 렌더링을 방해하지 않음.
 - young gc: 1~8MB 정도의 작은 From-Space와 생존자를 위한 To-Space가 있음.
 - old gc: concurrent marking => sweep => 종종 compact
 - 페이지: 256KB~1MB정도의 영역을 여러개로 나눠놓고 연결 리스트로 연결해서 사용함. => 파편 공간
- 그럼 차이는 뭐지?
 - g1gc는 young/old gc 크기와 주기가 훨씬 길다 => 단일 서버에 최적화. stw 200ms 타킷 디폴트
 - orinoco는 브라우저 60 fps 16ms 응답을 위해 1초에도 수십번 ~ 수백번씩도 gc가 일어날 수 있다. => 작고 빠르게.

보너스 - zgc는?

- JVM ZGC의 목적: 수십 테라바이트(TB)의 힙 메모리에서 STW를 1ms 이하로 억제. => 스루풋은 g1gc 대비 좀 떨어짐.
=> jvm에서는 이런 상황 자체가 올일이 없음.

마지막으로, 결국 각자의 고향의 영향을 많이 받았다는 것.

- V8은 브라우저 환경에서 성장했다.
 - 빠른 시작, 짧은 pause, 작은 단위의 반응성이 중요했다.
 - JS 실행은 기본적으로 event loop 중심이고, 병렬성은 Worker나 여러 프로세스로 확장하는 경우가 많다.
 - 그래서 Node.js 서버는 여러 프로세스/인스턴스를 띄워 scale-out하는 패턴과 잘 맞는다.
 - 대신 프로세스마다 JS heap은 기본적으로 분리되므로, 같은 객체를 여러 프로세스가 각각 들고 있는 비용이 생길 수 있다.
- HotSpot JVM은 장기 실행 서버 워크로드에서 강한 쪽으로 발전해왔다.
 - 실행 중 profile을 모아 JIT 최적화를 강하게 걸고, warmup 이후 높은 처리량을 노린다.
 - 하나의 큰 heap, 많은 thread, 긴 수명의 애플리케이션을 운영하고 관측하는 도구가 잘 발달했다.
 - G1, ZGC, Shenandoah 같은 GC는 큰 heap에서도 pause를 줄이려는 JVM의 방향성을 잘 보여준다.

디씨클 채용

감사합니다

원티드에서 디씨클 채용 공고도 많은 관심 부탁드립니다. (TS, Kotlin, Python)

기술 영역

Platform · RAG · Agent

그리고 Data Engineering, 검색, LLM 파이프라인

월간 방문자

30만명

트래픽

Peak TPS 1000+

평시 200-300 / 작지만 매월 성장중

데이터 처리량

최대 수십 TB

규모의 파이프라인 (프로덕트/젝트별 상이)